Lab 7: Fuzzing for Input Validation Bugs

Name: Paras Agarwal
SRN: PES1UG24CS316
Section: F
Assigned repository: SETAPESU26/L6_L7_11_amusement_park_ride_access (Lab 7)

---

1. Objective

The objective of this lab is to explore fuzz testing as a security and quality assurance technique by identifying and fixing bugs in the provided Python script using the Hypothesis fuzzing library. Instead of writing only fixed example tests, the properties of each function are described and Hypothesis generates many random, unexpected and malformed inputs to check them. The functions are then hardened against the invalid inputs that the fuzzer discovers.

2. Application Overview

The script `processor.py` belongs to an amusement park ride access system. Its `main()` function reads three values from the user and passes them to three functions:

- `validate_height_and_fastpass(rider_spec)` takes a string in the format `Height:FastPass` (for example `140:YES`). A rider of 120 cm or taller is Approved, otherwise Denied, and the FastPass value YES/NO decides between FastPass Priority and Standard Entry.
- `compute_dispatch_throughput(ride_metric_str)` takes a string in the format `riders/mins` (for example `48/2.5`) and returns the dispatch throughput in riders per minute, rounded to two decimal places.
- `resolve_safety_restraint_code(zone_code)` takes a ride zone code (for example `CST_01`) and maps its first three letters (CST, WTR, DRP or WHL) to the safety restraint used on that ride.

In the original version none of the functions validated their input. They assumed the input was always a string in the correct format with known values.

3. Fuzz Testing Approach

Hypothesis is a property-based testing library for Python. Each test is decorated with `@given` and receives inputs generated from strategies. When a property fails, Hypothesis shrinks the input to the smallest failing example and prints it. Hypothesis was installed together with pytest in a local virtual environment (`pip install hypothesis`, `pip install pytest`).

The three starter tests in `test_processor.py` were completed with the try/except pattern from the handout, and six more property tests were added. A `ValueError` or `TypeError` is treated as a controlled rejection of bad input. Any other exception (such as `KeyError`, `AttributeError` or `ZeroDivisionError`) or an impossible result is reported as a bug.

Strategies used:

- `st.text() | st.none()` for random Unicode strings, empty strings and None.
- `st.integers()` with bounds for valid heights, zero and negative heights.
- `st.floats()` for zero, negative, very large, nan and inf values in the throughput string.
- `st.sampled_from()` and `st.booleans()` for known FastPass values, known zone prefixes and upper/lower case.

Structured inputs were also built from these strategies (for example `f"{height}:{fastpass}"`) so that Hypothesis reaches deeper into each function instead of failing only at the format split. The nine tests and the property each one checks are listed below.

Test	Strategy	Property checked
`test_validate_height_and_fastpass_no_crash`	`st.text() | st.none()`	Never crashes; invalid input is rejected with ValueError or TypeError
`test_validate_height_and_fastpass_valid_spec`	`st.integers(min_value=1)`, `st.sampled_from`	Valid specs give the correct Approved/Denied decision and entry type (120 cm boundary)
`test_validate_height_and_fastpass_random_fastpass`	`st.integers`, `st.text()`	A FastPass value other than YES/NO is rejected, never crashes
`test_validate_height_and_fastpass_rejects_invalid_height`	`st.integers(max_value=0)`	Zero or negative heights raise ValueError
`test_compute_dispatch_throughput_safe`	`st.text() | st.none()`	Never crashes; any returned throughput is finite and non-negative
`test_compute_dispatch_throughput_valid_metric`	`st.integers`, `st.floats`	Valid riders/minutes give round(riders / minutes, 2)
`test_compute_dispatch_throughput_numeric_edge_cases`	`st.floats()` (zero, negative, nan, inf)	Either rejected with ValueError or a finite, non-negative result
`test_resolve_safety_restraint_code_safe`	`st.text() | st.none()`	Never crashes; any result is one of the four known restraints
`test_resolve_safety_restraint_code_known_zones`	`st.sampled_from`, `st.text`, `st.booleans`	Known prefixes (any case, any suffix) map to the correct restraint

4. Original Application Execution

The original `processor.py` was run with `python processor.py`. With the example inputs it works correctly. In the second run the FastPass value `MAYBE` was entered and the program crashed with an unhandled `KeyError`.

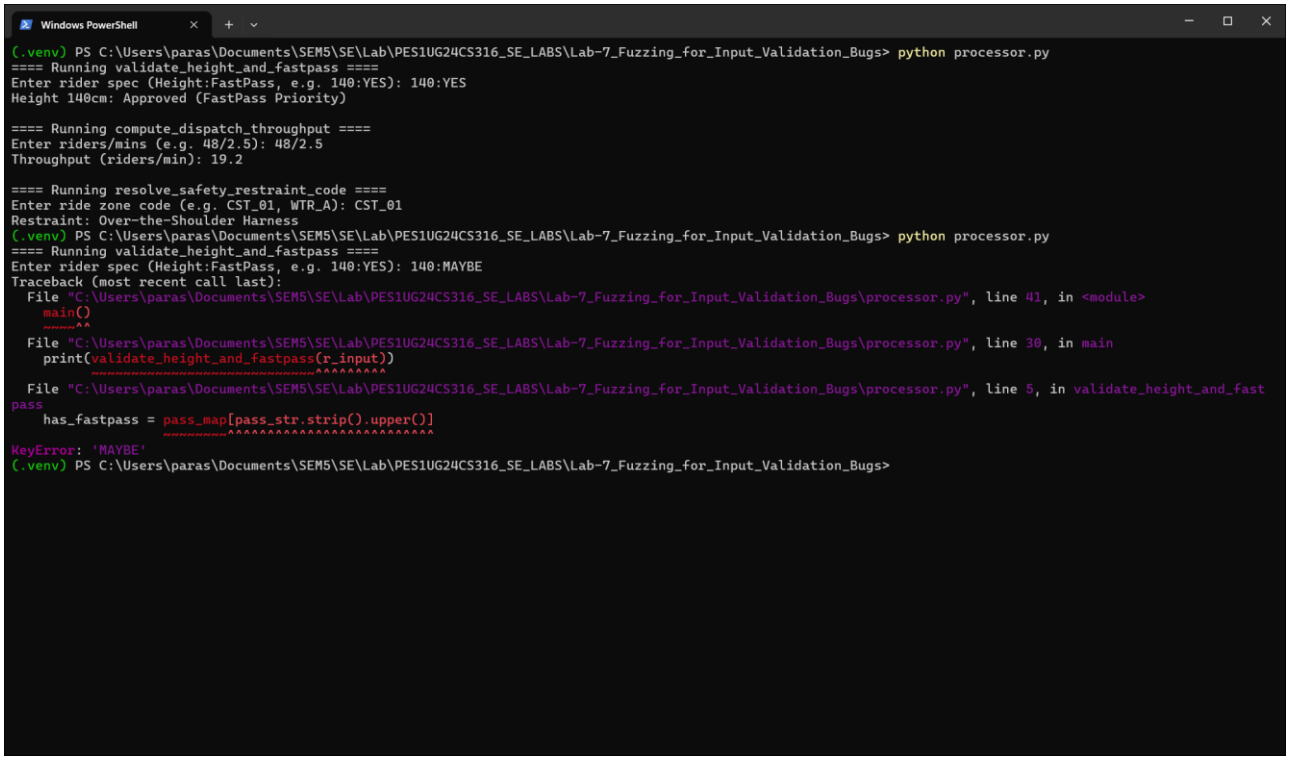


Figure 1: Original application output (valid run and crash on 140:MAYBE)

5. Initial Hypothesis Test Failures

The completed tests were run against the original application. The `--tb=no` option was added only to hide the long tracebacks so that the command and the full result fit on one screen; it does not change which tests run or their results.

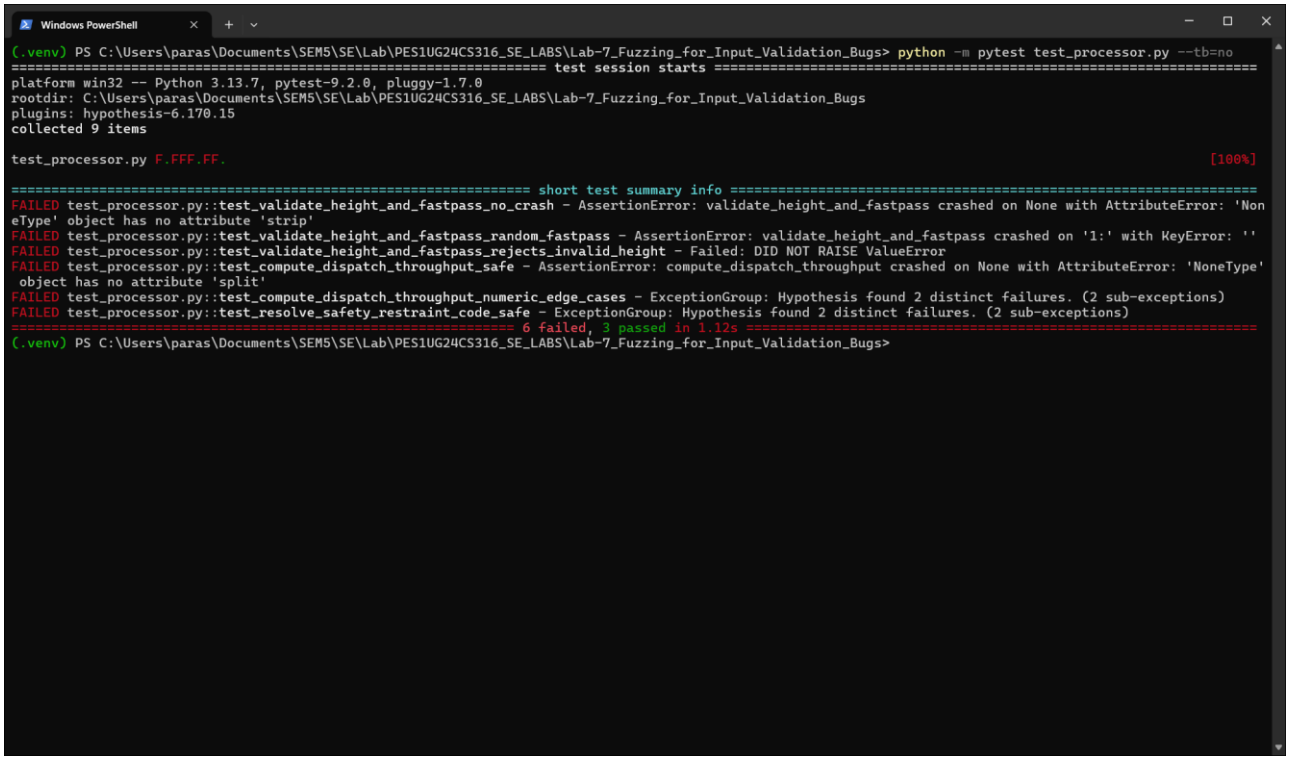


Figure 2: Hypothesis tests against the original application (6 failed, 3 passed)

Result: 6 failed, 3 passed. The three tests that use only valid inputs passed, which shows the functions work for well-formed input. The six failing tests exposed these minimized failing examples:

- `validate_height_and_fastpass(None)` crashed with AttributeError.
- `validate_height_and_fastpass('1:')` crashed with KeyError: '' (empty FastPass value).
- `validate_height_and_fastpass('0:YES')` did not raise ValueError and accepted a height of 0 cm.
- `compute_dispatch_throughput(None)` crashed with AttributeError.
- Hypothesis found 2 distinct failures in `compute_dispatch_throughput`: `'0.0/0.0'` crashed with ZeroDivisionError, and `'1.0/-1.0'` returned a negative throughput of -1.0.
- Hypothesis found 2 distinct failures in `resolve_safety_restraint_code`: `''` crashed with KeyError and `None` crashed with AttributeError.

6. Bugs Identified and Fixes Applied

The fixes were made in `processor.py` without changing the function names, parameters or the output for valid input. Invalid input now raises a `ValueError` (or `TypeError` for non-strings) with a clear message instead of an unexpected crash.

Bug discovered	Fix applied
All three functions crashed with `AttributeError` when given `None` (minimized input: `s=None`).	Each function now checks `isinstance(value, str)` first and raises a clear `TypeError` for non-string input.
`validate_height_and_fastpass` crashed with `KeyError` for a FastPass value other than YES/NO (minimized input: `'1:'`).	The FastPass value is checked against the allowed values and a `ValueError` ("FastPass must be YES or NO") is raised.
`validate_height_and_fastpass` accepted a height of zero or less (minimized input: `height=0`) and returned a normal result.	Heights that are not positive now raise `ValueError`.
`compute_dispatch_throughput` crashed with `ZeroDivisionError` when the cycle time was zero (minimized input: `'0.0/0.0'`).	Cycle minutes must be a finite number greater than zero, otherwise `ValueError` is raised.
`compute_dispatch_throughput` returned a negative throughput for negative values (minimized input: `'1.0/-1.0'` gave `-1.0`).	Riders must be finite and non-negative; the result is also checked with `math.isfinite`, so nan and inf values are rejected too.
`resolve_safety_restraint_code` crashed with `KeyError` for an empty or unknown zone code (minimized input: `s=''`).	The prefix is checked against the known restraint types and `ValueError` ("Unknown ride zone code") is raised.

In addition, both string formats are checked to have exactly one separator, so a missing or extra `:` or `/` gives a clear message instead of a Python unpacking error. `main()` now catches `ValueError` for each step and prints the message instead of stopping with a traceback.

7. Fixed Application Execution

The fixed application was run with the same valid inputs, which give the same output as before. A second run with `140:MAYBE`, `48/0` and `XYZ_9` shows that each invalid input is now reported with a clear message and the program finishes normally.

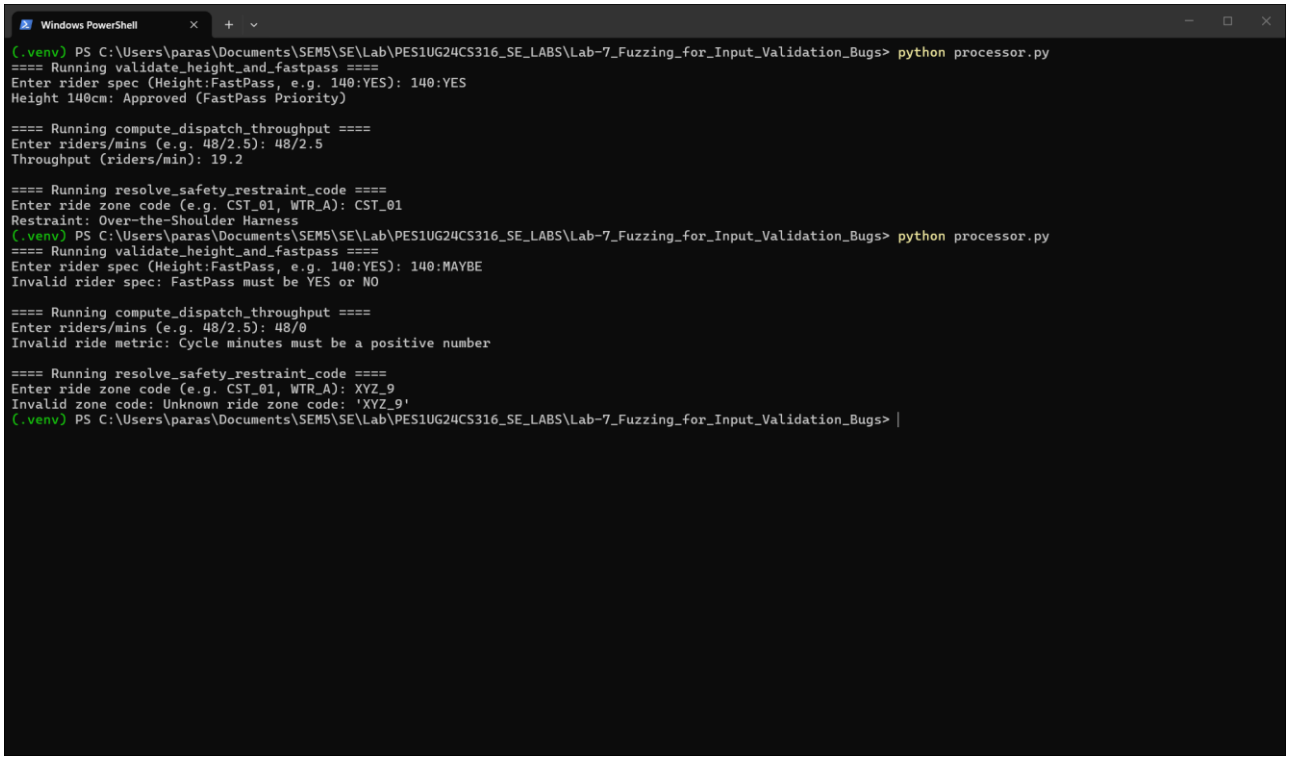


Figure 3: Fixed application output (valid run and invalid inputs handled)

8. Final Hypothesis Test Results

The same, unchanged test file was run again with `python -m pytest test_processor.py` against the fixed application.

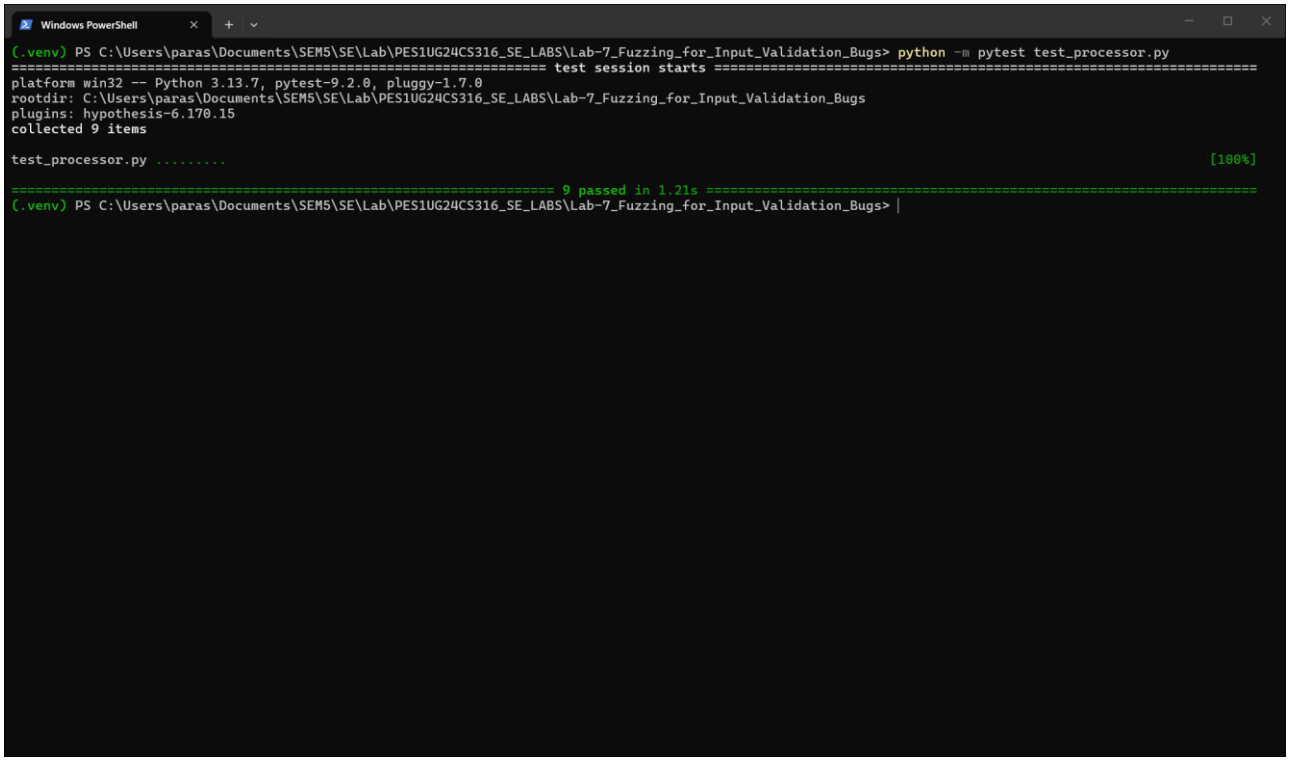


Figure 4: Hypothesis tests after the fix (9 passed)

Result: all 9 tests passed. The six tests that failed before now pass, and the three valid-input tests still pass, so the fixes did not change the correct behaviour. The tests were not weakened; only processor.py was changed between the two runs.

9. Conclusion

Fuzz testing with Hypothesis found six input-validation bugs across all three functions within seconds. These included crashes on None, empty or unknown values, a division by zero and results that were accepted even though they were impossible, such as a 0 cm height and a negative throughput. Several of these cases, such as an empty FastPass value or a 0.0/0.0 metric, are easy to miss when writing example-based tests by hand. Because Hypothesis shrinks every failure to a minimal example, each bug was simple to understand and fix. After type checks, format checks, value checks and clear error messages were added, the same property tests pass and the application handles invalid input safely while keeping its original behaviour for valid input.